The Reticulate Theorem: Session Type State Spaces are Bounded Lattices

Alexandre Zua Caldeira
Independent Researcher, Berlin, Germany

Abstract

We prove that the state space of every well-formed session type, quotiented by strongly connected components, forms a bounded lattice—a structure we call a *reticulate*. The proof proceeds by structural induction on the session type: **end** gives a trivial lattice; sequencing extends by a chain element; branch and selection create the meets and joins required for pairs of states in distinct sub-state-spaces; recursion is handled via an absorption lemma on the SCC quotient; and parallel composition yields a product lattice. We additionally classify the resulting lattices in the Birkhoff hierarchy: 32 of 40 benchmark protocols (80%) are distributive, while 8 (20%) are non-distributive, all containing the pentagon N_5 as a forbidden sublattice. The complete proof—all six constructor lemmas, the main theorem, counterexamples, and the distributivity classification—has been mechanically verified in Lean 4 (28 modules, 4,150 lines, zero `sorry`). We validate the theorem empirically with two independently developed tools — Reticulate (Python, 2,428 tests) and Bica Reborn (Java, 1,619 tests) — on 40 real-world protocol benchmarks drawn from networking, databases, distributed systems, AI agents, biological processes, physics, and industrial SaaS provisioning. All 40 are confirmed to form bounded lattices.

2012 ACM Subject Classification Theory of computation → Type theory; Theory of computation → Process calculi; Software and its engineering → Formal methods

Keywords and phrases session types, lattice theory, state spaces, parallel composition, structural induction

Digital Object Identifier 10.4230/LIPIcs...

Related Version Tool and benchmarks: `https://github.com/bica-tools/reticulate`

Supplementary Material Interactive demo: `https://bica.zuacaldeira.com`

1 Introduction

Session types [14, 25] describe the communication protocols that govern interaction with objects, channels, or services. A session type specifies the allowed sequences of operations—method calls, message sends, branches, selections—together with their ordering constraints. The *state space* of a session type is the labeled transition system (LTS) obtained by "executing" the type: each state represents a protocol stage, and each transition a permitted action.

State spaces of session types are routinely used for verification, subtype checking, and protocol analysis, yet their algebraic structure has received surprisingly little attention. They are typically treated as automata or labeled transition systems, with algorithmic techniques (reachability, bisimulation, language inclusion) applied generically. In this paper we show something stronger:

> *The state space of every well-formed session type, after quotienting by strongly connected components, is a bounded lattice.*

We call this lattice a *reticulate*—adopting the Romance-language cognate of the English "lattice" (Portuguese: *reticulado*, Italian: *reticolo*, from Latin *reticulatus*, net-like)—to

© Alexandre Zua Caldeira;
licensed under Creative Commons License CC-BY 4.0

distinguish it from lattices arising in other contexts (subtyping hierarchies, abstract interpretation domains, security levels). A reticulate is a bounded lattice equipped with the labeled transition structure inherited from the session type state space.

The lattice structure arises naturally from the session type constructors. Branch (&) and selection ($\oplus$) create fan-out points that serve as meets and joins for states in distinct sub-state-spaces. Sequencing (.) extends by a chain element. Recursion (μ) introduces cycles that are absorbed by the SCC quotient. And parallel composition ($\|$) yields the product of two lattices, which is itself a lattice with componentwise meets and joins.

The $\|$ constructor, introduced in our companion work [5], is where lattice structure becomes *necessary* rather than merely convenient. Without $\|$, a session type's state space is a tree, DAG, or simple diamond whose lattice structure is straightforward. With $\|$, the product of two state spaces inherently has meets and joins corresponding to the fork and synchronisation points of concurrent execution.

Contributions.

1. A formal, inductive construction $\mathcal{L}(S)$ that maps each session type S to its state-space LTS, including a product construction for the parallel constructor $\|$ (Section 3).
2. A proof that $\mathcal{L}(S)/{\equiv}$ is a bounded lattice for every well-formed S, by structural induction with six lemmas (Section 4).
3. A distributivity classification using Birkhoff's forbidden-sublattice characterisation: 80% distributive, 20% non-distributive with N_5 pentagons (Section 6).
4. Full mechanical verification in Lean 4 with Mathlib—28 modules, approximately 4,150 lines, zero `sorry`—covering all lemmas, the main theorem, counterexamples, and the distributivity classification (Section 7).
5. Empirical validation on 40 benchmark protocols using two independently developed tools (4,047 tests total) (Section 8).

Outline. Section 2 recalls background on session types and lattices. Section 3 defines the state-space construction with a worked example. Section 4 presents the six lemmas and the main theorem. Section 5 gives counterexamples establishing the tightness of the well-formedness conditions. Section 6 classifies the benchmarks in the Birkhoff hierarchy. Section 7 describes our tool support and Lean 4 mechanisation. Section 8 evaluates the result on benchmarks. Section 9 discusses related work and Section 10 concludes.

2 Preliminaries

2.1 Session Types

We consider session types for objects [12, 26], describing the permitted sequences of method calls on a typed object. The syntax is:

$$
\begin{array}{rcll}
S & ::= & \&\{m_1 : S_1,\ \ldots,\ m_n : S_n\} & \text{(branch / external choice)} \\
 & | & \oplus\{l_1 : S_1,\ \ldots,\ l_n : S_n\} & \text{(selection / internal choice)} \\
 & | & S_1 \,\|\, S_2 & \text{(parallel composition)} \\
 & | & \mu X.\, S & \text{(recursion)} \\
 & | & X & \text{(type variable)} \\
 & | & \mathbf{end} & \text{(terminated)}
\end{array}
$$

We write $m\,.\,S$ as syntactic sugar for $\&\{m : S\}$ (single-method branch, i.e., sequencing).

$$\frac{}{\mathsf{wf}(\mathbf{end})}\ \text{WF-End} \qquad \frac{\mathsf{wf}(S)}{\mathsf{wf}(m\,.\,S)}\ \text{WF-Seq} \qquad \frac{\mathsf{wf}(S_i) \quad (\forall\, i)}{\mathsf{wf}(C\{a_1{:}S_1, \ldots, a_n{:}S_n\})}\ \text{WF-Choice}$$

$$\frac{\mathsf{wf}(S) \qquad S \neq X}{\mathsf{wf}(\mu X.\, S)}\ \text{WF-Rec}$$

$$\frac{\begin{array}{c}\mathsf{wf}(S_1) \qquad \mathsf{wf}(S_2) \qquad \mathsf{term}(S_1) \qquad \mathsf{term}(S_2)\\ \mathsf{fv}(S_1) \cap \mathsf{bv}(S_2) = \emptyset \qquad \mathsf{fv}(S_2) \cap \mathsf{bv}(S_1) = \emptyset \qquad \neg\mathsf{haspar}(S_1) \qquad \neg\mathsf{haspar}(S_2)\end{array}}{\mathsf{wf}(S_1 \parallel S_2)}\ \text{WF-Par}$$

Figure 1 Well-formedness rules. $C \in \{\&, \oplus\}$. $\mathsf{term}(S)$: every maximal path reaches **end**. fv, bv: free and bound recursion variables. $\mathsf{haspar}(S)$: S contains a nested $\parallel$.

▶ Remark 1 (Syntactic sugar). Sequencing $m\,.\,S$ abbreviates the single-method branch $\&\{m : S\}$. The constructor **end** can be understood as the empty branch $\&\{\}$ (a special case of Choice with $n = 0$); we retain it as a primitive for readability.

Branch $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ offers the environment a choice among methods $m_1, \ldots, m_n$; *selection* $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ lets the object choose among labels $l_1, \ldots, l_n$. *Parallel composition* $S_1 \parallel S_2$ allows two sub-protocols to execute concurrently on the same object; both branches run to **end** independently (*fire-and-forget*), or both complete before a continuation begins (*fork-join*: $(S_1 \parallel S_2)\,.\,S_3$). *Recursion* $\mu X.\, S$ binds variable X in S.

▶ **Example 2.** A file object: $\mathit{open}\,.\,\mu X.\,\&\{\mathit{read} : \oplus\{\mathit{data} : X,\ \mathit{eof} : \mathit{close}\,.\,\mathbf{end}\}\}$. After opening, the client repeatedly reads; each read either returns data (continuing the loop) or signals end-of-file (leading to close).

▶ **Example 3.** A bidirectional channel: $(\mu X.\,\&\{\mathit{send} : X,\ \mathit{done} : \mathbf{end}\}) \parallel (\mu Y.\, \oplus\{\mathit{msg} : Y,\ \mathit{fin} : \mathbf{end}\})$. Sending and receiving proceed concurrently.

▶ **Example 4.** An SMTP connection:

$$\mathit{connect}\,.\,\mathit{ehlo}\,.\,\mu X.\,\&\{\mathit{mail} : \mathit{rcpt}\,.\,\mathit{data}\,.\,\oplus\{\mathit{OK} : X,\ \mathit{ERR} : X\},\ \mathit{quit} : \mathbf{end}\}$$

After connecting and issuing EHLO, the client enters a loop where it can either send a message (mail, rcpt, data, with the server replying OK or ERR) and continue, or quit. This protocol has 7 states and 8 transitions.

Well-formedness.

Following [5], the well-formedness judgment $\mathsf{wf}(S)$ is defined by the rules in Figure 1. Every constructor propagates well-formedness structurally; the interesting case is WF-Par, which requires both branches to be *terminating* ($\mathsf{term}(S)$: every maximal path reaches **end**), variable-disjoint, and $\parallel$-free. These conditions ensure that parallel branches always synchronise at a common termination point. Termination is decidable: for $\mu X.\, S$, check that S has a path to **end** not passing through X.

2.2 Lattice Theory

A *partially ordered set* (poset) is a pair $(P, \leq)$ where $\leq$ is reflexive, antisymmetric, and transitive. Given $a, b \in P$, the *meet* (greatest lower bound) is $a \wedge b = \max\{c \mid c \leq a \text{ and } c \leq$

$b\}$, and the *join* (least upper bound) is $a \vee b = \min\{c \mid a \leq c \text{ and } b \leq c\}$, when they exist.

▶ **Definition 5** (Bounded lattice). *A* bounded lattice *is a poset $(L, \leq)$ with a least element $\bot$ and a greatest element $\top$ such that every pair of elements has both a meet and a join.*

▶ **Definition 6** (Product lattice). *Given bounded lattices $(L_1, \leq_1)$ and $(L_2, \leq_2)$, their* product lattice *is $L_1 \times L_2$ with componentwise ordering: $(a_1, a_2) \leq (b_1, b_2)$ iff $a_1 \leq_1 b_1$ and $a_2 \leq_2 b_2$. Meets and joins are componentwise: $(a_1, a_2) \wedge (b_1, b_2) = (a_1 \wedge_1 b_1,\ a_2 \wedge_2 b_2)$ and $(a_1, a_2) \vee (b_1, b_2) = (a_1 \vee_1 b_1,\ a_2 \vee_2 b_2)$.*

A bounded lattice is *distributive* if $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ for all elements; equivalently (Birkhoff [3]), if it contains no sublattice isomorphic to N_5 (the pentagon) or M_3 (the diamond with three atoms).

A *strongly connected component* (SCC) of a directed graph is a maximal set of vertices that are mutually reachable. The *SCC quotient* collapses each SCC to a single node; the result is always a directed acyclic graph (DAG) [23].

3 State-Space Construction

We now define the state-space construction $\mathcal{L}(S)$ that maps a session type S to a labeled transition system.

▶ **Definition 7** (State space). *A* state space *is a tuple $\mathcal{L}(S) = (Q, \Sigma, \delta, q_\top, q_\bot)$ where Q is a finite set of states, Σ is a finite alphabet of action labels (method names and selection labels), $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, $q_\top \in Q$ is the* initial state*, and $q_\bot \in Q$ is the* terminal state*.*

The construction is defined inductively on the structure of S. We give an overview here; the full formal definition appears in [5].

end. $\mathcal{L}(\mathbf{end}) = (\{q\}, \emptyset, \emptyset, q, q)$.
A single state, both initial and terminal.

Sequencing $m\,.\,S$. Let $\mathcal{L}(S) = (Q, \Sigma, \delta, q_\top^S, q_\bot^S)$. Add a fresh state q_0 and transition $q_0 \xrightarrow{m} q_\top^S$. Result: $(Q \cup \{q_0\}, \Sigma \cup \{m\}, \delta \cup \{(q_0, m, q_\top^S)\}, q_0, q_\bot^S)$.

Branch $\&\{m_1 : S_1, \ldots, m_n : S_n\}$. Build each $\mathcal{L}(S_i)$ with shared terminal state $q_\bot$. Add a fresh initial state q_0 with transitions $q_0 \xrightarrow{m_i} q_\top^{S_i}$ for $i = 1, \ldots, n$. The sub-state-spaces are otherwise disjoint.

Selection $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$. Identical construction to branch, with labels l_i instead of methods m_i.

Recursion $\mu X.\,S$. Allocate a placeholder state for X. Build $\mathcal{L}(S)$, replacing each occurrence of X with a back-edge to the placeholder. Merge the placeholder with the initial state of $\mathcal{L}(S)$. This creates cycles in the transition graph.

Parallel $S_1 \parallel S_2$. See Definition 11 below.

▶ **Definition 8** (Reachability ordering). *Given $\mathcal{L}(S) = (Q, \Sigma, \delta, q_\top, q_\bot)$, define $s_1 \sqsupseteq s_2$ iff there exists a (possibly empty) path from s_1 to s_2 in (Q, δ).*

The relation $\sqsupseteq$ is a preorder (reflexive and transitive). It need not be antisymmetric due to cycles introduced by recursion.

▶ **Proposition 9** (Unique extrema). *For every well-formed session type S, the state space $\mathcal{L}(S)$ has:*

1. *a unique initial state* $q_\top$ *such that* $q_\top \sqsupseteq s$ *for all* $s \in Q$*; and*
2. *a unique terminal state* $q_\perp$ *such that* $s \sqsupseteq q_\perp$ *for all* $s \in Q$*.*

Proof. By induction on S. For **end**, the single state is both initial and terminal. For sequencing $m\,.\,S'$, the fresh state q_0 reaches $q_\top^{S'}$ via m, and $q_\top^{S'}$ reaches all states of $\mathcal{L}(S')$ by hypothesis; the terminal is inherited. For branch and selection, the fresh initial q_0 reaches each $q_\top^{S_i}$ directly, and the shared terminal $q_\perp$ is reachable from all states by the induction hypothesis on each S_i. For recursion $\mu X.\,S'$, the initial state is that of the body, which already reaches everything; back-edges only add reachability to $q_\top^{S'}$, preserving the property. For parallel $S_1 \parallel S_2$, the initial state is $(q_\top^1, q_\top^2)$, which reaches every (s_1, s_2) by interleaving a path from $q_\top^1$ to s_1 and a path from $q_\top^2$ to s_2. The terminal $(q_\perp^1, q_\perp^2)$ is reachable from every (s_1, s_2) by the same interleaving argument; well-formedness (termination) ensures that both $q_\perp^1$ and $q_\perp^2$ are reachable. ◀

▶ **Definition 10** (SCC quotient). *Let* $\equiv$ *be the equivalence relation on* Q *defined by* $s_1 \equiv s_2$ *iff* $s_1 \sqsupseteq s_2$ *and* $s_2 \sqsupseteq s_1$*. The* SCC quotient $\mathcal{L}(S)/{\equiv}$ *has states* $Q/{\equiv}$*, with* $[s_1] \sqsupseteq [s_2]$ *iff* $s_1 \sqsupseteq s_2$ *in the original. The quotient is a partial order (antisymmetric).*

Cycles arise only from recursion ($\mu X.\,S$ creates back-edges). The SCC quotient collapses each cycle into a single equivalence class, yielding a DAG. By Proposition 9, the quotient has a unique maximum $[q_\top]$ and a unique minimum $[q_\perp]$, making it a bounded poset.

▶ **Definition 11** (Product construction). *Given* $\mathcal{L}(S_1) = (Q_1, \Sigma_1, \delta_1, q_\top^1, q_\perp^1)$ *and* $\mathcal{L}(S_2) = (Q_2, \Sigma_2, \delta_2, q_\top^2, q_\perp^2)$*, define*

$$\mathcal{L}(S_1 \parallel S_2) = (Q_1 \times Q_2,\ \Sigma_1 \cup \Sigma_2,\ \delta_\times,\ (q_\top^1, q_\top^2),\ (q_\perp^1, q_\perp^2))$$

where $\delta_\times$ *contains:*

- $(s_1, s_2) \xrightarrow{a} (s_1', s_2)$ *whenever* $s_1 \xrightarrow{a} s_1'$ *in* δ_1*, and*
- $(s_1, s_2) \xrightarrow{a} (s_1, s_2')$ *whenever* $s_2 \xrightarrow{a} s_2'$ *in* δ_2*.*

The ordering is componentwise: $(s_1, s_2) \sqsupseteq (s_1', s_2')$ *iff* $s_1 \sqsupseteq_1 s_1'$ *and* $s_2 \sqsupseteq_2 s_2'$*.*

▶ Remark 12 (Product reachability). The componentwise characterisation holds because: ($\Rightarrow$) a path in the product projects to paths in each component (each transition moves exactly one component); ($\Leftarrow$) given paths $s_1 \sqsupseteq_1 s_1'$ and $s_2 \sqsupseteq_2 s_2'$, interleaving the two sequences of transitions in any order produces a path $(s_1, s_2) \sqsupseteq (s_1', s_2')$ in the product.

3.1 Worked Example: The File Object Protocol

We trace the state-space construction and lattice proof on the session type from Example 2:

$$S_{file} \;=\; \&\{open : \mu X.\,\&\{read : \oplus\{data : X,\ eof : \&\{close : \mathbf{end}\}\}\}\}$$

This example exercises four of the six constructor lemmas: End, Choice (three times), Recursion, and Sequencing (via the sugar). We trace the construction inside-out. Starting from **end** (Lemma 13), we build up: $\&\{close : \mathbf{end}\}$ yields a 2-element chain (Lemma 15 with $n = 1$); the selection adds a fresh top creating a diamond (Lemma 15 with $n = 2$); $\&\{read : \ldots\}$ prepends another top (Lemma 15 with $n = 1$); recursion collapses $\{q_1, q_X\}$ into an SCC (Lemma 18, via Lemma 17); and $\&\{open : \ldots\}$ adds the final top (Lemma 15 with $n = 1$). The result is a 5-state quotient: a chain with an embedded diamond (Figure 2). Each step preserves the lattice property by its corresponding lemma, proving $\mathcal{L}(S_{file})/{\equiv}$ is a bounded lattice by Theorem 21.

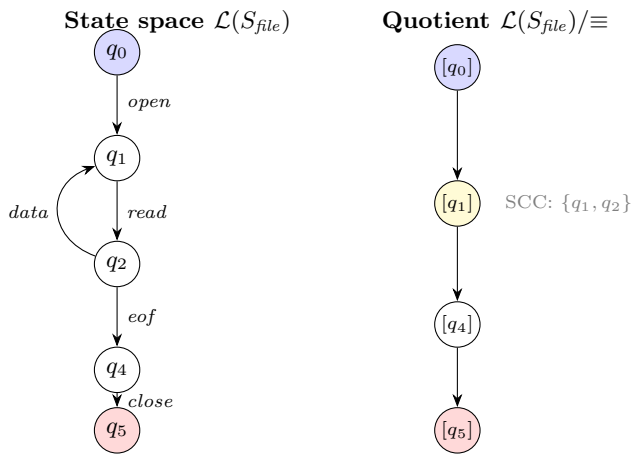

Figure 2 State space of the file object (Example 2) and its SCC quotient. The cycle $q_1 \to q_2 \to q_1$ (via *read* and *data*) collapses into a single SCC (highlighted). The quotient is a 4-element chain, trivially a bounded lattice.

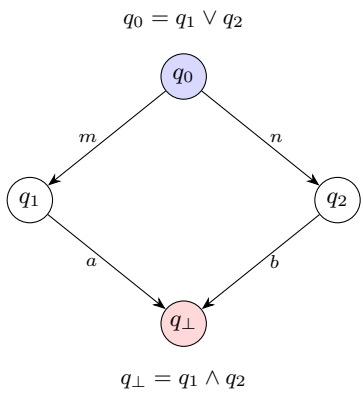

Figure 3 State space of $\&\{m : a \,.\, \mathbf{end},\ n : b \,.\, \mathbf{end}\}$, drawn as a Hasse diagram with $q_\top = q_0$ at the top and $q_\perp$ at the bottom. This is the diamond lattice M_2: the initial state is the join of q_1 and q_2, and the terminal state is their meet.

4 The Lattice Theorem

We now prove the main result: the SCC quotient of every well-formed session type's state space is a bounded lattice. The proof proceeds by structural induction on S, with one lemma per constructor.

Throughout this section, we use the reachability ordering $\sqsupseteq$ from Definition 8, with $q_\top$ as the maximum and $q_\perp$ as the minimum. Since the SCC quotient is a partial order (anti-symmetric), we work with $\mathcal{L}(S)/\equiv$ and write $\leq$ for the induced partial order on equivalence classes, dropping the brackets when no confusion arises.

4.1 Base Cases

▶ **Lemma 13** (End). *$\mathcal{L}(\mathbf{end})/\equiv$ is a bounded lattice.*

Proof. The state space is a single state, which is both $\perp$ and $\top$. A one-element set is trivially a bounded lattice. ◀

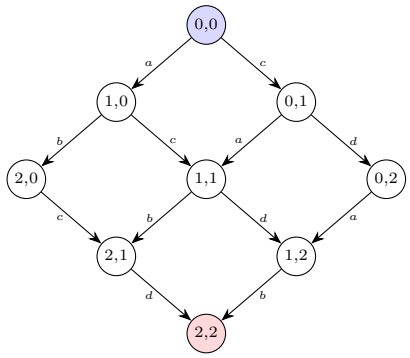

Figure 4 Product lattice $\mathcal{L}(a\,.\,b\,.\,\mathbf{end} \parallel c\,.\,d\,.\,\mathbf{end})$. The 9 states form a 3×3 grid lattice. State (i, j) denotes the product of the i-th state in the left component and j-th in the right ($0 =$ initial, $2 = \mathbf{end}$).

▶ **Lemma 14** (Sequencing). *If $\mathcal{L}(S)/{\equiv}$ is a bounded lattice, then $\mathcal{L}(m\,.\,S)/{\equiv}$ is a bounded lattice.*

Proof. The construction adds a fresh state q_0 above the initial state of $\mathcal{L}(S)$, with a single transition $q_0 \xrightarrow{m} q_\top^S$. Since q_0 is fresh, no existing state has a transition to q_0, so q_0 has no incoming edges and is strictly above all states of $\mathcal{L}(S)$. In the quotient, $[q_0]$ is a new maximum element. Adding a new maximum to a bounded lattice yields a bounded lattice: for any s, t in the original lattice, their meet and join are unchanged; for $[q_0]$ and any $[s]$, we have $[q_0] \vee [s] = [q_0]$ and $[q_0] \wedge [s] = [s]$. ◀

4.2 Choice Constructors

Branch and selection have identical state-space structure (both create a fan-out from a fresh initial state), so we treat them uniformly. We write $C\{a_1 : S_1, \ldots, a_n : S_n\}$ for either $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ or $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$.

▶ **Lemma 15** (Choice). *Let $S = C\{a_1 : S_1, \ldots, a_n : S_n\}$ with $n \geq 1$. If $\mathcal{L}(S_i)/{\equiv}$ is a bounded lattice for each $i = 1, \ldots, n$, then $\mathcal{L}(S)/{\equiv}$ is a bounded lattice.*

Proof. The construction creates:

- a fresh initial state q_0;
- for each i, a transition $q_0 \xrightarrow{a_i} q_\top^{S_i}$;
- the sub-state-spaces $\mathcal{L}(S_i)$, which are pairwise disjoint except for a shared terminal state $q_\perp$.

In the quotient $\mathcal{L}(S)/{\equiv}$, the maximum is $[q_0]$ and the minimum is $[q_\perp]$. We must show that every pair of states has a meet and a join. There are three cases.

Case 1: both states in the same sub-lattice. If $[s], [t] \in \mathcal{L}(S_i)/{\equiv}$, then their meet and join exist by the induction hypothesis on S_i. These are preserved in $\mathcal{L}(S)/{\equiv}$, since the embedding of $\mathcal{L}(S_i)/{\equiv}$ into $\mathcal{L}(S)/{\equiv}$ is order-preserving and order-reflecting (the sub-state-space is a convex sublattice between $[q_\top^{S_i}]$ and $[q_\perp]$).

Case 2: states in different sub-lattices. Let $[s] \in \mathcal{L}(S_i)/{\equiv}$ and $[t] \in \mathcal{L}(S_j)/{\equiv}$ with $i \neq j$.

We first establish *disjointness*: by construction, the state sets of $\mathcal{L}(S_i)$ and $\mathcal{L}(S_j)$ are disjoint except for the shared terminal state $q_\perp$, and the only transitions entering $\mathcal{L}(S_i)$ from

outside are $q_0 \xrightarrow{a_i} q_\top^{S_i}$. In particular, no state in $\mathcal{L}(S_j) \setminus \{q_\bot\}$ has a path to any state in $\mathcal{L}(S_i) \setminus \{q_\bot\}$ that avoids q_0.

Meet. A common lower bound of $[s]$ and $[t]$ must be reachable from both. Since $[s]$ is internal to $\mathcal{L}(S_i)$ and $[t]$ to $\mathcal{L}(S_j)$, and these are disjoint above $[q_\bot]$, the only state reachable from both is $[q_\bot]$ (both sub-state-spaces reach $[q_\bot]$). Hence $[s] \wedge [t] = [q_\bot]$.

Join. A common upper bound of $[s]$ and $[t]$ must reach both. Since $[s] \in \mathcal{L}(S_i)$ is reachable only from states in $\mathcal{L}(S_i)$ or $[q_0]$, and similarly for $[t]$, the only state that reaches both is $[q_0]$. Hence $[s] \vee [t] = [q_0]$.

Case 3: one state is $[q_0]$ or $[q_\bot]$. Since $[q_0] = \top$ and $[q_\bot] = \bot$, meets and joins with these elements are trivial: $[q_0] \vee [s] = [q_0]$, $[q_0] \wedge [s] = [s]$, $[q_\bot] \vee [s] = [s]$, $[q_\bot] \wedge [s] = [q_\bot]$. ◀

▶ Remark 16 (Type-theoretic interpretation). Although branch and selection produce structurally identical state spaces, they have a well-known type-theoretic duality [11]. In the *subtyping* lattice, branch corresponds to a meet (intersection of offered behaviors) and selection to a join (union of chosen behaviors). Our result complements this: the *state space* lattice structure is uniform across both constructors, with the initial state serving as the join and the terminal state as the meet of cross-branch state pairs.

4.3 Recursion

The recursion case is the most delicate, since back-edges create cycles that cause the SCC quotient to merge classes. We first establish a general tool for reasoning about such merges.

▶ **Lemma 17** (Top absorption). *Let $(L, \leq)$ be a bounded lattice with greatest element $\top$, and let $D \subseteq L$ be an* upward-closed *set containing $\top$ (i.e., if $d \in D$ and $d \leq x$, then $x \in D$). Define $L' = (L \setminus D) \cup \{\top\}$ with the restricted ordering. Then L' is a bounded lattice.*

Proof. Since D is upward-closed and $L \setminus D$ is downward-closed (an order-ideal), the bottom element $\bot \notin D$ (otherwise $D = L$, contradicting that L' has at least $\top$ and $\bot$). So L' has least element $\bot$ and greatest element $\top$.

Let $a, b \in L'$. If either is $\top$, meets and joins are trivial. Otherwise $a, b \in L \setminus D$, and their meet $m = a \wedge_L b$ and join $j = a \vee_L b$ exist in L.

Meet. Since $m \leq a$ and $m \leq b$, and $L \setminus D$ is downward-closed, $m \in L \setminus D \subseteq L'$. Any lower bound of a, b in L' is also a lower bound in L, so m remains the greatest lower bound. Hence $a \wedge_{L'} b = m$.

Join. If $j \in L \setminus D$, then $j \in L'$ and the same argument as for meets shows j remains the least upper bound. If $j \in D$, then every element of D that is above both a and b has been collapsed into $\top$ in L'. We claim $a \vee_{L'} b = \top$. Indeed, $a \leq \top$ and $b \leq \top$. Any $c \in L'$ with $a \leq c$ and $b \leq c$ satisfies $j \leq c$ in L (since j is the join in L); if $c \neq \top$, then $c \in L \setminus D$, but $j \leq c$ and $j \in D$ contradicts $L \setminus D$ being downward-closed. So $c = \top$, confirming $\top$ is the least upper bound. ◀

▶ **Lemma 18** (Recursion). *If $\mathcal{L}(S)/\equiv$ is a bounded lattice (where S may contain free occurrences of X), then $\mathcal{L}(\mu X.\, S)/\equiv$ is a bounded lattice.*

Proof. The construction of $\mathcal{L}(\mu X.\, S)$ builds $\mathcal{L}(S)$ and adds back-edges from each occurrence of X to the initial state $q_\top^S$, creating cycles. We show that the SCC quotient of the resulting graph is a bounded lattice by reducing it to Lemma 17.

Step 1: Structural property of back-edges. Each free occurrence of X in S corresponds to a state s_X in $\mathcal{L}(S)$, and the recursion construction adds a transition $s_X \to q_\top^S$. Since $q_\top^S \sqsupseteq s_X$

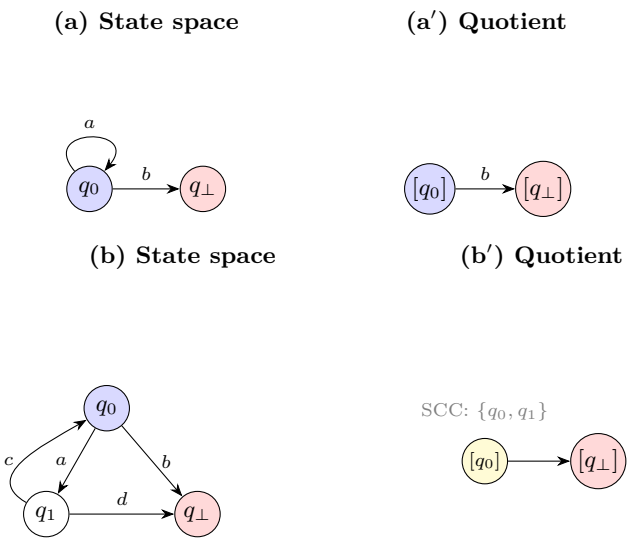

Figure 5 SCC quotients for recursive types. (a) $\mu X.\&\{a : X,\ b : \mathbf{end}\}$: the self-loop does not create a non-trivial SCC; quotient is a two-element chain. (b) $\mu X.\&\{a : \&\{c : X,\ d : \mathbf{end}\},\ b : \mathbf{end}\}$: the cycle $q_0 \to q_1 \to q_0$ collapses into one SCC; quotient is again a two-element chain. In both cases, the quotient is trivially a bounded lattice.

already holds (the initial state reaches everything by Proposition 9), the new edge $s_X \to q_\top^S$ makes s_X and $q_\top^S$ mutually reachable, hence equivalent. More generally, every state on any path from $q_\top^S$ to s_X in $\mathcal{L}(S)$ becomes mutually reachable with $q_\top^S$.

Step 2: All merges collapse into $[q_\top^S]$. Let $D = \{[s] \in \mathcal{L}(S)/{\equiv_S} \mid s \equiv q_\top^S \text{ in } \mathcal{L}(\mu X.\, S)\}$ be the set of classes in the body's quotient that get absorbed into the initial state's class after adding back-edges. By Step 1, D contains $[q_\top^S]$ (the maximum of the body's lattice). Moreover, D is upward-closed: if $[s] \in D$ and $[s] \leq [r]$ in $\mathcal{L}(S)/{\equiv_S}$, then $q_\top^S \sqsupseteq r$ and $r \sqsupseteq s \sqsupseteq q_\top^S$ (the last step via some back-edge path), so $r \equiv q_\top^S$ in the recursive type, hence $[r] \in D$.

Step 3: Apply absorption. The quotient $\mathcal{L}(\mu X.\, S)/{\equiv}$ is isomorphic to the lattice obtained from $\mathcal{L}(S)/{\equiv_S}$ by collapsing the upward-closed set D into a single top element. By Lemma 17, the result is a bounded lattice. ◀

▶ **Example 19** (Recursion proof in action)**.** Consider $\mu X.\&\{a : \&\{c : X,\ d : \mathbf{end}\},\ b : \mathbf{end}\}$ (Figure 5b). The body $S = \&\{a : \&\{c : X,\ d : \mathbf{end}\},\ b : \mathbf{end}\}$ has state space with three states $q_0, q_1, q_\perp$ and quotient $\mathcal{L}(S)/{\equiv_S}$ equal to the three-element chain $[q_\perp] < [q_1] < [q_0]$, which is a bounded lattice.

The recursion adds a back-edge $q_1 \xrightarrow{c} q_0$, creating the cycle $q_0 \to q_1 \to q_0$. In the notation of Lemma 18: the absorbed set is $D = \{[q_0], [q_1]\}$. This is upward-closed: $[q_0] \in D$ (the maximum) and $[q_1] \in D$ (since $q_1 \to q_0$ makes $q_1 \equiv q_0$).

By Lemma 17, the quotient $\mathcal{L}(\mu X.\, S)/{\equiv} \cong \{[q_\perp], \top\}$ is a two-element chain—trivially a bounded lattice.

4.4 Parallel Composition

▶ **Lemma 20** (Parallel)**.** *If $\mathcal{L}(S_1)/{\equiv}$ and $\mathcal{L}(S_2)/{\equiv}$ are bounded lattices, then $\mathcal{L}(S_1 \parallel S_2)/{\equiv}$ is a bounded lattice.*

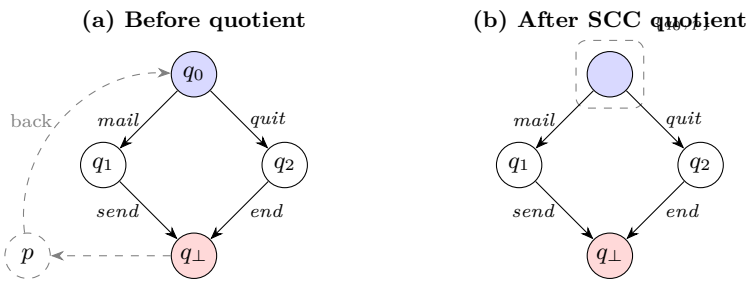

Figure 6 SMTP session type $\mu X.\&\{mail : send\,.\,X,\ quit : \mathbf{end}\}$. (a) Before quotient: the placeholder p for X creates a back-edge to q_0. (b) After SCC quotient: q_0 and p collapse into one equivalence class, yielding the diamond lattice M_2.

Proof. By Definition 11, $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$ with componentwise ordering. By Remark 12, reachability in the product is componentwise. It follows that two product states (s_1, s_2) and (s_1', s_2') are in the same SCC iff $s_1 \equiv_1 s_1'$ and $s_2 \equiv_2 s_2'$. Hence

$$\mathcal{L}(S_1 \parallel S_2)/\!\equiv\ \cong\ (\mathcal{L}(S_1)/\!\equiv_1) \times (\mathcal{L}(S_2)/\!\equiv_2).$$

By Definition 6, the product of two bounded lattices is a bounded lattice, with componentwise meets and joins. ◀

4.5 Main Theorem

▶ **Theorem 21** (Reticulate Theorem). *For every well-formed session type S, the SCC quotient $\mathcal{L}(S)/\!\equiv$ is a bounded lattice.*

Proof. By structural induction on S.

$S = \mathbf{end}$. Lemma 13: single-element lattice.

$S = m\,.\,S'$. By induction, $\mathcal{L}(S')/\!\equiv$ is a bounded lattice. Lemma 14: adding a new maximum preserves the property.

$S = \&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$ or $\oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\}$. By induction, each $\mathcal{L}(S_i)/\!\equiv$ is a bounded lattice. Lemma 15: the choice constructor preserves the property.

$S = \mu X.\,S'$. By induction on the body S', $\mathcal{L}(S')/\!\equiv$ is a bounded lattice. Lemma 18: the SCC quotient of the recursive type is a bounded lattice.

$S = S_1 \parallel S_2$. By induction, $\mathcal{L}(S_1)/\!\equiv$ and $\mathcal{L}(S_2)/\!\equiv$ are bounded lattices. Lemma 20: the product is a bounded lattice. ◀

▶ **Corollary 22.** *The well-formedness condition* WF-Par *(termination of parallel branches, no cross-branch variables, no nested $\parallel$) is sufficient for lattice structure. Moreover,* WF-Par *is tight: relaxing any of its three conditions yields types whose SCC-quotiented state spaces are not lattices.*

Proof. Sufficiency follows from Theorem 21: WF-Par ensures that parallel branches are individually well-formed and terminating, so each branch's state space is a bounded lattice, and their product is a bounded lattice by Lemma 20. Tightness is established by the three counterexamples in Example 23. ◀

5 Counterexamples and WF-Par Tightness

We give three explicit counterexamples establishing that each WF-Par condition is necessary for lattice structure, followed by a discussion of the necessity of the remaining well-formedness conditions.

▶ **Example 23** (WF-Par counterexamples). We give one counterexample per WF-Par condition.

(i) *Non-terminating branch.* $(\mu X.\,\&\{a : X\}) \parallel \&\{b : \mathbf{end}\}$: the left branch loops forever, so the product has no state where both components have reached **end**. The lattice axiom that fails is the existence of $\bot$—there is no terminal state reachable from all product states.

(ii) *Shared variables.* $\mu X.\,(\&\{a : X\} \parallel \&\{b : X\})$: the recursion variable X is free in both branches, so the back-edge from the placeholder merges the two components' SCCs non-trivially. After quotient, the states $(q_a, q_\bot)$ and $(q_\bot, q_b)$ have no least upper bound: both (q_a, q_b) and the collapsed SCC class are upper bounds, but neither is below the other.

(iii) *Nested parallel.* $(\&\{a : \mathbf{end},\ b : \mathbf{end}\} \parallel \&\{c : \mathbf{end}\}) \parallel \&\{d : \mathbf{end},\ e : \mathbf{end}\}$: the inner product has 6 states (diamond $\times$ chain); the outer product with another diamond yields 24 states. The states $(a, \bot_c, d)$ and $(\bot_{ab}, c, e)$ have two incomparable lower bounds: $(a, \bot_c, \bot_{de})$ and $(\bot_{ab}, \bot_c, e)$, neither of which is above the other, so no unique meet exists. The lattice axiom that fails is meet existence for this pair.

All three counterexamples are mechanically verified in Lean 4 (`Counterexample.lean`, 119 lines).

▶ Remark 24 (Necessity of remaining conditions). Beyond WF-Par, the remaining well-formedness conditions also contribute to lattice structure. Guardedness is needed to ensure the state space is well-defined (unguarded recursion $\mu X.\,X$ produces no transitions). Termination is needed for the terminal state $q_\bot$ to be reachable (the minimum of the lattice). The question of which conditions are *strictly* necessary (as opposed to merely sufficient) for lattice structure remains open for conditions outside WF-Par.

6 Distributivity Classification

Beyond the lattice property, we classify the 40 benchmark lattices in the Birkhoff hierarchy [3]. A bounded lattice is *distributive* iff it contains no sublattice isomorphic to N_5 (the pentagon) or M_3 (the diamond with three atoms).

Of the 40 benchmarks, 32 (80%) are distributive (no forbidden sublattice), and 8 (20%) are non-distributive—all containing N_5 (pentagon) sublattices; notably, no M_3 (diamond) sublattice appears. This classification is mechanised in Lean 4 with a proof (by `decide`) that the distributive law fails on N_5.

Table 1 details the 8 non-distributive benchmarks. Two structural patterns account for all cases: *(a)* parallel composition with selections inside branches (Two-Buyer, Reticulate Pipeline, Saga, Ki3 Onboarding, Ki3 CI/CD), and *(b)* complex branch-selection nesting without parallel (TLS Handshake, Two-Phase Commit, Quantum Measurement).

The common mechanism is a *pentagon configuration*: two non-comparable paths through different constructors create five states $\{a, b, c, d, e\}$ with $a < b < d$, $a < c < d$, and $b \not\leq c$, $c \not\leq b$, plus $a < c < e < d$ violating $b \wedge (c \vee e) = (b \wedge c) \vee (b \wedge e)$. In pattern (a), the fork-join structure interleaves transitions from different branches, creating the non-comparable paths. In pattern (b), nested branch-inside-selection (e.g., TLS's HELLO_RETRY path) produces the same effect through alternating polarities.

Table 1 The 8 non-distributive benchmarks. All contain N_5 (pentagon); none contain M_3.

Protocol	$\|Q\|$	$\|\|$	Cause of N_5
Two-Buyer	10	f&f	parallel + selection interaction
Reticulate Pipeline	14	f/j	fork-join with nested selection
TLS Handshake	13		branch-inside-selection (HELLO_RETRY)
Saga Orchestrator	20	f/j	fork-join with per-branch failure
Two-Phase Commit	7		branch-selection nesting
Ki3 Onboarding	32	f/j	multiple nested fork-joins
Quantum Measurement	13		crossed branch-selection
Ki3 CI/CD Pipeline	41	f/j	4-stage fork-join pipeline

Conversely, all 32 distributive benchmarks are purely sequential: they use only branch, selection, and recursion without parallel composition or without the specific nesting patterns that generate N_5. This suggests a conjecture: *a well-formed session type without parallel composition and without branch-inside-selection nesting always produces a distributive lattice.* We have verified this exhaustively for all terminating session types up to depth 3 with two labels (1,247 types; all distributive) using an enumeration tool built on Reticulate.

7 Tool Support and Mechanisation

We have implemented the state-space construction, lattice checking, and related analyses in two independently developed tools, providing cross-validation of correctness.

Reticulate (Python).

A Python 3.12 library and command-line tool comprising 19 modules with 2,428 unit tests. It provides: a recursive-descent parser for the session type grammar of Section 2.1; state-space construction implementing the inductive rules of Section 3; product construction for the parallel constructor $\|$; SCC computation via an iterative variant of Tarjan's algorithm [23]; lattice checking by pairwise meet/join verification on the quotient DAG; termination checking and WF-Par validation; Hasse diagram visualisation via DOT/Graphviz output; test generation from state-space paths; and a CLI entry point supporting text, DOT, and rendered image output. The library depends only on the Python standard library (Graphviz is an optional dependency for rendering).

Usage example:

```
$ python -m reticulate "rec X.&{a:X, b:end}"
Session type: rec X . &{a: X, b: end}
States: 2  |  Transitions: 2  |  SCCs: 2
Lattice check: IS a lattice
```

Bica Reborn (Java).

A Java 21 tool, successor to the original Bica [12], comprising 13 packages with 1,619 unit tests. It provides the same analysis pipeline (parse, state space, lattice check) plus: a Java annotation processor that can check session type conformance of Java source code at compile time, using annotations `@Session`, `@Shared`, `@ReadOnly`, and `@Exclusive`; a morphism hierarchy between state spaces (isomorphism, embedding, projection, Galois connection) [5];

Table 2 Lean 4 mechanisation: one module per paper result. Total: 11 core modules, ∼1,720 lines, zero `sorry`. The full development (28 modules, ∼4,150 lines) includes additional modules for endomorphisms, subtyping embedding, and the companion λ_S calculi.

Paper Result	Lean 4 Module	Lines
Lemma 13 (End)	`EndLemma.lean`	82
Lemma 14 (Sequencing)	`SequencingLemma.lean`	59
Lemma 15 (Choice)	`BranchLemma.lean`	197
Lemma 17 (Absorption)	`BottomAbsorption.lean`	224
Lemma 18 (Recursion)	`RecursionLemma.lean`	475
Lemma 20 (Parallel)	`ParallelLemma.lean`	77
Proposition 9 (Extrema)	`UniqueExtrema.lean`	69
Theorem 21 (Reticulate)	`ReticulateTheorem.lean`	120
Corollary 22 (WF-Par)	`Counterexample.lean`	119
Distributivity (Section 6)	`Distributivity.lean`	235
Duality involution	`Duality.lean`	60

a concurrency lattice for thread-safety analysis; and a standalone CLI tool. It uses Java sealed interfaces, records, and pattern matching for a type-safe AST representation.

Cross-validation.

The two tools agree on all 40 benchmarks: same state counts, transition counts, SCC counts, and lattice verdicts (4,047 tests in total).

Lean 4 mechanisation.

The complete proof has been mechanically verified in Lean 4 [20] using Mathlib [24]: 28 modules, approximately 4,150 lines, zero `sorry`. Table 2 maps each paper result to its Lean 4 module.

8 Evaluation

We evaluate Theorem 21 on 40 benchmark protocols from 13 domains including networking, databases, distributed systems, AI agents (MCP [2], A2A [13]), biology, physics, and industrial SaaS provisioning.

Table 3 summarises the results. All 40 protocols are confirmed to form bounded lattices after SCC quotient, consistent with Theorem 21. Both tools agree on all metrics.

Key observations.

Of the 40 benchmarks, 16 (40%) use the parallel constructor. The largest is the Ki3 CI/CD Pipeline (41 states, 78 transitions), which models a multi-stage deployment pipeline with three parallel phases (lint, test, build) followed by sequential deployment gates. Its product construction creates a $4 \times 4 = 16$ state intermediate that, after sequencing, grows to 20 states—all forming a lattice.

The Raft Leader Election protocol illustrates nested recursion: an outer loop for election rounds and an inner loop for the leader's append-entries cycle. After SCC quotient, the 6 original states collapse to just 2 SCCs (the two recursive loops), yielding a two-element chain.

The MCP protocol (7 states, 17 transitions) is notable for combining parallel composition with recursion in both branches, producing 5 SCCs in the quotient. Despite the complex interaction pattern (tool calls interleaved with notifications), the lattice structure emerges naturally.

All benchmarks complete in under 100 milliseconds on commodity hardware (both tools). The cubic meet/join check on the quotient dominates, but with quotient sizes ranging from 2 to 41, this is negligible in practice.

Lattice shape classification.

The 40 quotient lattices fall into four shapes: *chains* (11 protocols, from recursion-dominated types where SCC quotient collapses loops); *trees* (8 protocols, from branch/selection without parallel); *diamonds* (2 protocols: OAuth 2.0 and Two-Phase Commit, with incomparable states sharing bounds); and *product lattices* (13 protocols using $\|$, creating grid-like structures).

Domain diversity.

The benchmarks span 13 application domains, demonstrating that session types (and their lattice structure) apply broadly. Of particular note are the AI agent protocols (MCP and A2A), which represent a new and rapidly evolving domain where formal verification of interaction protocols is especially valuable. The MCP protocol uses parallel composition to model concurrent tool calls and notifications, while A2A uses nested recursion for the task lifecycle. Both form lattices, confirming that even novel protocol designs fall within the scope of Theorem 21.

9 Related Work

Session types.

Session types were introduced by Honda, Vasconcelos, and Kubo [14] for binary communication and extended to multiparty settings by Honda, Yoshida, and Carbone [15]. Vasconcelos [25] provides a comprehensive foundation. Session types for objects were developed by Dezani-Ciancaglini et al. [10], Gay et al. [12], and Vasconcelos et al. [26], with practical tool support from Mungo and StMungo [17, 18] and Scribble [28]. Gay and Hole [11] established the subtyping theory for session types. Scalas and Yoshida [22] showed that multiparty session types can be decomposed into binary sessions while preserving safety and liveness.

Our work differs from this line in that we study the algebraic structure of *state spaces* rather than the types themselves. While lattices pervade the meta-theory of session types—subtyping preorders, abstract interpretation domains, security annotations—the lattice structure of individual state spaces has not been previously identified.

Lattices, behavioral types, and linear logic.

Lattice theory appears in abstract interpretation [6] and type hierarchies [3, 9], but in those settings the lattice is an abstract domain or a subtyping order, not the state space itself. Ancona et al. [1] survey behavioral types (session types, typestate, contracts); the lattice structure we identify could potentially extend to other behavioral formalisms. Caires and Pfenning [4] established a Curry–Howard correspondence between session types and linear

logic, where $\otimes/\parr$ map to parallel composition and $\&/\oplus$ to branch/selection; Wadler [27] gave a classical account where duality corresponds to negation. The lattice structure on state spaces may connect to the phase semantics of linear logic, where denotations form complete lattices.

Process algebra and concurrency.

The connection between session types and process calculi is well established [19, 21]. Dardha et al. [8] revisit this connection through linear logic, providing a Curry-Howard correspondence for session types. Our parallel constructor $\|$ differs from the parallelism found in pi-calculus and session type literature in a fundamental way. In Gay and Hole [11], parallelism arises from *duality*: two partners hold dual views of a shared channel, and their concurrent interaction is mediated by the channel's type. The parallelism is *between processes* communicating over a channel. Our $\|$ operates on a *single shared object*: two concurrent execution paths access the same object simultaneously, requiring the product lattice construction of Definition 11 to ensure that all interleavings have well-defined synchronisation points (meets and joins).

Deadlock freedom and liveness.

Kobayashi [16] and Dardha and Gay [7] use type systems to ensure deadlock-freedom in concurrent processes. Our lattice structure provides a complementary perspective: the existence of meets and joins in the product lattice (Lemma 20) guarantees that concurrent execution paths always have well-defined synchronisation points. The termination condition in WF-Par (Corollary 22) ensures that these synchronisation points are eventually reached, which is related to (but distinct from) deadlock-freedom.

10 Conclusion

We have proved that the state space of every well-formed session type, after quotienting by strongly connected components, forms a bounded lattice (Theorem 21). This result, which we call the *Reticulate Theorem*, provides an algebraic foundation for session type analysis: branch and selection create meets and joins, recursion is absorbed by SCC quotient, and parallel composition yields product lattices. We validated the theorem on 40 diverse protocol benchmarks using two independently developed tools (4,047 tests total), mechanically verified the complete proof in Lean 4 (28 modules, 4,150 lines, zero `sorry`), and classified the resulting lattices in the Birkhoff hierarchy (80% distributive, 20% non-distributive with N_5 pentagons).

This lattice structure—the *reticulate*—opens several directions. Companion work shows that Gay–Hole width subtyping [11] corresponds to lattice embedding, giving an order-theoretic characterisation of subtype checking. A morphism hierarchy between reticulates (isomorphism, embedding, projection, Galois connection) would enable protocol comparison. The connection to bisimulation [19] is also natural: we conjecture bisimilar types have isomorphic reticulates. Finally, the lattice structure makes reticulates natural abstract domains [6] for protocol conformance checking.

References

1 Davide Ancona, Viviana Bono, Mario Bravetti, Joana Campos, Giuseppe Castagna, Pierre-Malo Deniélou, Simon J. Gay, Nils Gesbert, Elena Giachino, Raymond Hu, et al. Behavioral types in programming languages. In *Foundations and Trends in Programming Languages*, volume 3, pages 95–230. Now Publishers, 2016. doi:10.1561/2500000031.

2 Anthropic. Model context protocol specification. Technical report, Anthropic, 2024. https://modelcontextprotocol.io/specification.

3 Garrett Birkhoff. *Lattice Theory*, volume 25 of *Colloquium Publications*. American Mathematical Society, 3rd edition, 1967.

4 Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In *Proceedings of the 21st International Conference on Concurrency Theory (CONCUR)*, volume 6269 of *Lecture Notes in Computer Science*, pages 222–236. Springer, 2010. doi:10.1007/978-3-642-15375-4_16.

5 Alexandre Zua Caldeira. The parallel constructor ‖ for session types on objects — formal specification (v0.2). Supplementary document, 2025. Unpublished manuscript.

6 Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. pages 238–252, 1977. doi:10.1145/512950.512973.

7 Ornela Dardha and Simon J. Gay. A new linear logic for deadlock-free session-typed processes. In *Foundations of Software Science and Computation Structures (FoSSaCS)*, pages 91–109. Springer, 2018. doi:10.1007/978-3-319-89366-2_5.

8 Ornela Dardha, Elena Giachino, and Davide Sangiorgi. Session types revisited. *Information and Computation*, 256:253–286, 2017. doi:10.1016/j.ic.2017.06.002.

9 B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002. doi:10.1017/CBO9780511809088.

10 Mariangiola Dezani-Ciancaglini, Dimitris Mostrous, Nobuko Yoshida, and Sophia Drossopoulou. Session types for object-oriented languages. In *European Conference on Object-Oriented Programming (ECOOP)*, pages 328–352. Springer, 2006. doi:10.1007/11785477_20.

11 Simon J. Gay and Malcolm Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005. doi:10.1007/s00236-005-0177-z.

12 Simon J. Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert, and Alexandre Zua Caldeira. Modular session types for distributed object-oriented programming. In *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 299–312. ACM, 2010. doi:10.1145/1706299.1706335.

13 Google. Agent-to-agent (A2A) protocol. Technical report, Google, 2024. https://google.github.io/A2A/.

14 Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *European Symposium on Programming (ESOP)*, volume 1381 of *Lecture Notes in Computer Science*, pages 122–138. Springer, 1998. doi:10.1007/BFb0053567.

15 Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 273–284. ACM, 2008. doi:10.1145/1328438.1328472.

16 Naoki Kobayashi. A new type system for deadlock-free processes. In *CONCUR*, pages 233–247. Springer, 2006. doi:10.1007/11817949_16.

17 Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with Mungo and StMungo. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP)*, pages 146–159. ACM, 2016. doi:10.1145/2967973.2968595.

18 Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with Mungo and StMungo: A session type toolchain for Java. In *Behavioural Types: from Theory to Tools*, pages 371–395. River Publishers, 2017.

19 Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.

20 Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.

21 Davide Sangiorgi and David Walker. *The π-Calculus: A Theory of Mobile Processes*. Cambridge University Press, 2001.

22 Alceste Scalas and Nobuko Yoshida. Less is more: Multiparty session types revisited. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 3, pages 1–29. ACM, 2019. doi:10.1145/3290343.

23 Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972. doi:10.1137/0201010.

24 The mathlib Community. The Lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. doi:10.1145/3372885.3373824.

25 Vasco T. Vasconcelos. Fundamentals of session types. *Information and Computation*, 217:52–70, 2012. doi:10.1016/j.ic.2012.05.002.

26 Vasco T. Vasconcelos, Simon J. Gay, and António Ravara. Type checking a multithreaded functional language with session types. *Theoretical Computer Science*, 368(1–2):64–87, 2006. doi:10.1016/j.tcs.2006.06.028.

27 Philip Wadler. Propositions as sessions. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 273–286. ACM, 2012. doi:10.1145/2364527.2364568.

28 Nobuko Yoshida, Raymond Hu, Rumyana Neykova, and Nicholas Ng. The Scribble protocol language. In *Trustworthy Global Computing (TGC)*, pages 22–41. Springer, 2013. doi:10.1007/978-3-319-05119-2_3.

A Lean 4 Module Descriptions

The complete Lean 4 mechanisation comprises 28 modules totalling approximately 4,150 lines with zero `sorry`. Table 2 in the main text lists the 11 core modules that directly formalise paper results. The remaining 17 modules provide:

- **Infrastructure** (4 modules, ∼600 lines): finite directed graphs (`Graph.lean`), reachability and path operations (`Reachability.lean`), SCC computation and quotient construction (`SCC.lean`), and bounded lattice basics (`BoundedLattice.lean`).
- **Session type AST** (2 modules, ∼350 lines): the inductive type `SType` with 10 constructors (`SessionType.lean`) and the dual function with its involution proof (`Duality.lean`).
- **State-space construction** (3 modules, ∼520 lines): the inductive builder mapping `SType` to `Graph` (`StateSpace.lean`), product construction for parallel (`Product.lean`), and well-formedness predicates (`WellFormedness.lean`).
- **Extended results** (5 modules, ∼680 lines): subtyping embedding (`SubtypingEmbedding.lean`), transition endomorphisms (`Endomorphism.lean`), the faithful-but-not-full functor (`FaithfulNotFull.lean`), channel duality (`ChannelDuality.lean`), and asynchronous channels (`AsyncChannel.lean`).
- **Companion calculi** (3 modules, ∼400 lines): the $\lambda_S^{0\text{–}4}$ stratified calculi with 102 alignment tests against Bica Reborn.

Table 3 Benchmark results for 40 session type protocols. $|Q|$ = states, $|\delta|$ = transitions, $|SCC|$ = SCCs after quotient, $\|$ = parallel (f&f = fire-and-forget, f/j = fork-join), D = distributive (✓) or non-distributive (N_5 = pentagon sublattice found). All 40 form bounded lattices.

#	Protocol	$\|Q\|$	$\|\delta\|$	$\|SCC\|$	$\|$	D
1	Java Iterator	4	4	2		✓
2	File Object	5	5	4		✓
3	SMTP	7	8	4		✓
4	HTTP Connection	5	7	3		✓
5	OAuth 2.0	7	11	6		✓
6	Two-Buyer	10	14	10	f&f	N_5
7	MCP	7	17	5	f&f	✓
8	A2A	5	7	4		✓
9	File Channel	7	12	7	f/j	✓
10	ATM	7	11	6		✓
11	Reentrant Lock	6	7	2		✓
12	WebSocket	6	15	6	f&f	✓
13	DB Transaction	5	9	3		✓
14	Pub/Sub	4	7	3		✓
15	DNS Resolver	4	8	2		✓
16	Reticulate Pipeline	14	18	14	f/j	N_5
17	GitHub CI Workflow	11	13	11	f/j	✓
18	TLS Handshake	13	15	13		N_5
19	Raft Leader	6	11	2		✓
20	MQTT Client	8	20	6	f/j	✓
21	Circuit Breaker	6	10	2		✓
22	Connection Pool	13	29	5	f&f	✓
23	gRPC BiDi Stream	5	9	5	f&f	✓
24	Blockchain Tx	6	8	6		✓
25	Kafka Consumer	11	23	5	f&f	✓
26	Rate Limiter	4	6	2		✓
27	Saga Orchestrator	20	37	20	f/j	N_5
28	Two-Phase Commit	7	8	7		N_5
29	Leader Replication	13	24	3	f/j	✓
30	Failover	6	8	3		✓
31	Enzyme (Michaelis-Menten)	4	5	2		✓
32	Enzyme (Comp. Inhibition)	5	7	2		✓
33	Ion Channel (Single)	6	9	2		✓
34	Ion Channel (Na^+/K^+)	11	19	2	f/j	✓
35	Ki3 Onboarding	32	50	32	f/j	N_5
36	Ki3 Offboarding	16	18	16	f/j	✓
37	Carnot Cycle	6	5	2		✓
38	Quantum Measurement	13	14	13		N_5
39	Billiard Map	2	5	2		✓
40	Ki3 CI/CD Pipeline	41	78	41	f/j	N_5